CSE 211 — Theory of Computation

Pushdown Automata, PDA–CFG Equivalence and the CFL Pumping Lemma

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Atif Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — PDA design, conversions and the CFL pumping lemma
Lesson 5 — The Pushdown Automaton: Definition and Idea — ε-NFA + stack, 7-tuple, ww^R, diagrams
Lesson 6 — Instantaneous Descriptions and Traces — ID, ⊢, reachable IDs, reading a given PDA
Lesson 7 — Designing PDAs — the TF design catalogue with verified machines
Lesson 8 — Final State vs Empty Stack — L(P) vs N(P), both conversions
Lesson 9 — CFG ↔ PDA — grammar → PDA, PDA → grammar [pXq]
Lesson 10 — CFL Pumping Lemma, DPDA, Hierarchy — proof, non-CF proofs, deterministic PDAs, Chomsky hierarchy
Quick Notes — PDA — revision bullets + key diagrams

NOTE Slide numbers refer to the PDF pages of Lecture - 3 (pda).pdf (136 pages; beamer builds). Atif Sir uses Hopcroft notation P = (Q, Σ, Γ, δ, q0, Z0, F) with δ(q, a, X) ∋ (p, γ) (γ replaces X; γ's first symbol ends on top); diagrams are labelled a, X/γ. Hakim Sir's Sipser notation (a, X → Y) appears in his PDA note and in the machine listings here. Every PDA and conversion in the solutions was run by a simulator against the language definition (strings up to length 7–8). Practice provenance: Actual = taken from a TF paper; Adapted = TF question with changed details; Constructed = new question in TF style.

Exam Map — What the TF Papers Ask

Session	PDA / CFL questions
2023-24	Q7 Example of a non-context-free language; prove it with the CFL pumping lemma (15) · Q8 PDA for {a^n b^m c^m d^n | n ≥ 1, m ≥ 1} (15)
2022-23	Q8 PDA for {a^i b^j c^k | i = j or i = k} (15) · Q9 State and prove the pumping lemma for CFLs (CNF parse-tree height theorem given) (15)
2021-22	Q7 PDA for ARITH = legal arithmetic expressions over {a,+,∗,(,)} (15) · Q8 Example of a non-context-free language; prove it with the CFL pumping lemma (15)
2020 (Jan term)	Q8 PDA for {a^i b^j c^k | i = j or i = k} (15)
2019-20	Q8 PDA for ARITH = legal arithmetic expressions over {a,+,∗,(,)} (15) · Q10(a) Final-state PDAs and empty-stack PDAs accept the same languages (8)
2017-18	Q8 PDA for {a^n b^n c^m d^m | n ≥ 1, m ≥ 1} (15)
2016-17	Q6(a) PDA for {0^n 1^m | n ≤ m ≤ 2n}; show it accepts 00111 and rejects 0011111 (15) · Q7(b) PDA for {ww^R | w ∈ {0,1}∗} (10) · Q7(c) {a^n b^n c^n | n ≥ 0} is not context-free (10)
2015-16	Q5(a) Draw the transition diagram of PDA P = ({q,p},{0,1},{Z0,X},δ,q,Z0,{p}) (8⅓) · Q5(b) All reachable IDs of that PDA from (q,w,Z0) for 0011, 0101, 1010 (15) · Q6(a) PDA for {w ∈ {a,b}∗ : equal numbers of a's and b's} (13) · Q6(b) From PF (final state) build PN (empty stack) for the same L — definitions of both (10⅓)
2014-15	Q6(b) {a^n b^j c^k : k = jn} is not context-free (7) · Q6(c) PDA for {wcw^R | w ∈ {a,b}∗}; trace bbcbb by configurations (9⅓) · Q7(a) For any PDA P there is a CFG G with L(P) = L(G) (7)
2013-14	Q1(a) Convert E→E+T|T, T→T×F|T/F|F, F→(E)|a to a PDA (7) · Q1(b) PDA for {a^i b^j c^k : i = j or j = k} (7) · Q1(c) Prove: for any PDA P there is a CFG G with L(P) = L(G) (9⅓) · Q2(c) Not CFL: {0^n1^n0^n1^n}, {0^n#0^2n#0^3n} (9⅓)
2012-13	Q1(a) Formal PDA definition; PDA for S→BS|A, A→0A|ε, B→BB1|2 (10) · Q1(b) Prove a language is CFL iff some PDA accepts it (13⅓) · Q2(b) Formal CFL pumping lemma; show {a^n b^n c^n | n ≥ 0} is not CFL (13⅓) · Q4(a) PDA for {a^i b^j c^k : i + j = k} (8) · Q4(c) PDA for odd-length strings over {a,b} with middle symbol a (8⅓)
2011-12	Q3(c) PDA for odd-length strings over {a,b} with middle symbol a (7) · Q4(a) PDA for {a^i b^j c^k : i + j = k} (7) · Q4(b) Convert S→aTXb, T→XTS|ε, X→a|b to a PDA (8)
2010-11	Q12(c) Prove: if a PDA recognizes L then L is context-free (11) · Q12(d) Informal description + state diagram of a PDA for S→TX, T→0T0|1T1|#X, X→0X|1X|ε (5⅓) · Q13(a) Formally describe a PDA for {w ∈ {0,1}∗ | w = w^R} (8) · Q13(b) Convert the PDA of 13(a) into an equivalent CFG (7⅓) · Q13(c) Run the given PDA (q1…q4, $ marker) on 1001 and 0101; what does it do? (8)
2009-10	Q10(a) Formal PDA definition; PDAs for {0^n1^n | n ≥ 1}, equal 0s and 1s, palindromes over {0,1} (14) · Q10(b) PDA P({p,q},…,δ: (q,0,Z)→XZ, …): (i) transition diagram, (ii) convert to an empty-stack PDA for the same language (9⅓) · Q11(a) Convert S→AS|A, A→0A|1B|1, B→0B|0 to a PDA accepting by empty stack (5) · Q11(b) Convert PDA δ(q,1,Z)={(q,XZ)}, δ(q,0,X)={(p,X)}, δ(p,1,X)={(p,ε)} to a CFG (13) · Q11(c) Deterministic PDA with an example (5⅓)
2008-09	Q1(i) PDA for {a^n b^2n | n > 0} or prove impossible (10) · Q1(ii) PDA for {a^n b^2n c^n | n ≥ 0} or prove impossible (13⅓)
2007-08	Q5 PDA for {a^i c^k (bc)^i | i, k ≥ 0} (10) · Q6 Justify: L has a PDA accepting by final state iff it has one accepting by empty stack (10)
2006-07	Q8 Define PDA with an example; what is an instantaneous description (ID)? (10) · Q9 Transition diagram of a PDA accepting palindromes of all lengths over {0,1} (10)

EXAM FOCUS ★★★ since 2019-20: (1) **a PDA design** every year (15 marks) — i = j or i = k (2022-23, Jan-2020), a^nb^mc^md^n (2023-24), a^nb^nc^md^m (2017-18), ARITH (2021-22, 2019-20); (2) a **CFL pumping-lemma proof** every year (2023-24, 2022-23 'state and prove', 2021-22); (3) 2019-20 Q10(a): final state vs empty stack. Older papers (2006-2016) loved CFG ↔ PDA conversions (both directions), 'trace by configurations' and 'all reachable IDs' — solved here in full. Lesson 7 first, then 10.

Lesson 5 — The Pushdown Automaton: Definition and Idea

Slides 1–55

1. Why a Stack (sl. 2–9)

A PDA is essentially an ε-NFA plus a stack that can be read, pushed and popped only at the top. It recognises exactly the context-free languages (sl. 2, 8). {ww^R} is CF (S → aSa | bSb | ε) but a left-to-right device must remember the first half to check it in reverse — nondeterministically guess the middle, push the first half, then pop-and-compare (sl. 3–4). Balanced parentheses: count +1 per '(' and −1 per ')'; a counter is a one-symbol stack (sl. 6). A PDA can store unbounded information but only LIFO; {0^n1^n2^n} is not recognisable by any PDA (sl. 8–9).

2. One Move (sl. 12–19)

In one transition the PDA (1) consumes the input symbol used (none if ε); (2) goes to a new state; (3) replaces the top stack symbol by a string γ: γ = ε pops; γ = X leaves the stack; γ = Y replaces X by Y; γ = YZ replaces X by Z and pushes Y on top (first symbol on top).

3. Formal Definition (sl. 26–32, 40–43)

P = (Q, Σ, Γ, δ, q0, Z0, F)
δ(q, a, X) = a finite set of pairs (p, γ), a ∈ Σ ∪ {ε}

Q states; Σ input symbols; Γ stack alphabet; q0 start; Z0 ∈ Γ the start symbol (the stack initially holds one Z0); F accepting states. **No mixing and matching** (sl. 40–43): from δ(q, a, X) = {(p, YZ), (r, ε)} you may go to p and replace X by YZ, or go to r and pop X — not p and pop X.

4. Example: ww^R (sl. 20–39, 44–55)

P = ({q0, q1, q2}, {0,1}, {0,1,Z0}, δ, q0, Z0, {q2}); q0 = 'middle not yet seen', q1 = 'past the middle, matching', q2 = 'done'. Z0 marks the bottom so that after popping w we still have a symbol to take the move into q2 (sl. 33–34).

```
1. δ(q0, 0, Z0) = {(q0, 0Z0)}   δ(q0, 1, Z0) = {(q0, 1Z0)}        first symbol: push, keep Z0 below
2. δ(q0, a, X) = {(q0, aX)} for a ∈ {0,1}, X ∈ {0,1}                 keep pushing
3. δ(q0, ε, Z0) = {(q1, Z0)}  δ(q0, ε, 0) = {(q1, 0)}  δ(q0, ε, 1) = {(q1, 1)}   guess: middle
reached
4. δ(q1, 0, 0) = {(q1, ε)}    δ(q1, 1, 1) = {(q1, ε)}               match and pop
5. δ(q1, ε, Z0) = {(q2, Z0)}                                         stack back to Z0: accept
```

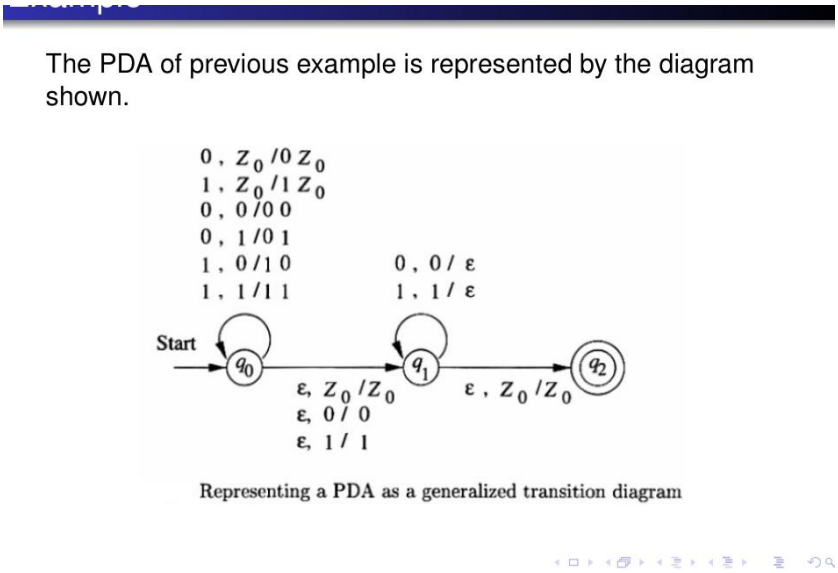


Figure 5.1 — The ww^R PDA as a diagram: arcs labelled a, X/γ (sl. 49)

MEMORIZE Diagram arc 'a, X/α' from q to p means δ(q, a, X) contains (p, α). The diagram does not show the start stack symbol — by convention it is Z0 (sl. 45–48).

Practice — Lesson 5

(a) Actual — 2006-07 Q8, 2009-10 Q10(a), 2012-13 Q1(a) (definition parts).

(a) Define a pushdown automaton with an example. What is an instantaneous description? (10)

(b) Formally define a PDA and design PDAs for {0^n1^n | n ≥ 1}, for strings with equal numbers of 0s and 1s, and for palindromes over {0, 1}. (14)

✓ Solution

(a) Definition: 7-tuple above. Example: ww^R PDA of §4. ID: Lesson 6.

(b) Definition: §3. Designs in Lesson 7 (0^n1^n: n ≥ 1 version is the Lesson 7 machine without the ε-branch; equal counts; palindromes).

Lesson 6 — Instantaneous Descriptions and Traces

Slides 56–86

1. ID and the Turnstile (sl. 56–67)

An **ID** is a triple (q, w, γ): state, remaining input, stack contents (top at the LEFT of γ, bottom at the right). If δ(q, a, X) contains (p, α) then for all w ∈ Σ∗, β ∈ Γ∗: (q, aw, Xβ) ⊢ (p, w, αβ). The rest of the input and the stack below X are carried along. ⊢∗ means zero or more moves.

Three principles (sl. 81–84): a legal computation stays legal if you (1) append the same extra input to every ID, (2) put the same extra symbols below the stack in every ID, (3) delete an unconsumed input tail. 'Data P never looks at cannot affect its computation.' (A finite automaton could use IDs (q, w) too, sl. 85–86.)

2. The Tree of Guesses on 1111 (sl. 68–80)

```
Accepting branch (middle after two 1s):
(q0,1111,Z0) ⊢ (q0,111,1Z0) ⊢ (q0,11,11Z0) ⊢ (q1,11,11Z0) ⊢ (q1,1,1Z0) ⊢ (q1,ε,Z0) ⊢ (q2,ε,Z0)
Other guesses: middle at the start → (q1,1111,Z0) ⊢ (q2,1111,Z0) — input not consumed, not
accepting;
middle after one 1 → (q1,111,1Z0) ⊢ (q1,11,Z0) — Z0 on top with input left; only ε to q2: not
accepting.
```

Exam Approach — Reading a Given PDA

1. Translate each transition into words (push / pop / replace / change state).
2. Draw the diagram (states as circles, arcs 'a, X/γ', start arrow, double circle for F).
3. Trace with IDs; to list ALL reachable IDs expand every nondeterministic choice (breadth first) and stop at IDs with no move.
4. State the language and check one member and one non-member.

Practice — Lesson 6

(a) Actual — 2015-16 Q5(a)–(b). (b) Actual — 2010-11 Q13(c). (c) Actual — 2014-15 Q6(c) trace.

(a) PDA P = ({q, p}, {0,1}, {Z0, X}, δ, q, Z0, {p}) with (i) δ(q,0,Z0) = {(q,XZ0)}, (ii) δ(q,0,X) = {(q,XX)}, (iii) δ(q,1,X) = {(q,X)}, (iv) δ(q,ε,X) = {(p,ε)}, (v) δ(p,ε,X) = {(p,ε)}, (vi) δ(p,1,X) = {(p,XX)}, (vii) δ(p,1,Z0) = {(p,ε)}. Draw the transition diagram. (8⅓) For the initial ID (q, w, Z0) show all reachable IDs when w is (i) 0011, (ii) 0101, (iii) 1010. (15)

(b) Run the PDA q1 −ε,ε→$→ q2 (loops 0,ε→0; 1,ε→1) −ε,ε→ε→ q3 (loops 0,0→ε; 1,1→ε) −ε,$→ε→ q4 (q1, q4 accepting) on 1001 and 0101; describe the language. (8)

(c) Trace the computation of your PDA for {wcw^R} that accepts bbcbb, by configurations. (9⅓)

✓ Solution

(a) Diagram: q −0, Z0/XZ0→ q (loop), q −0, X/XX→ q, q −1, X/X→ q, q −ε, X/ε→ p, p −ε, X/ε→ p (loop), p −1, X/XX→ p (loop), p −1, Z0/ε→ p (loop); start q, accepting p.

```
w = 0011: 17 reachable IDs, accepted: yes
  (q, 0011, Z0)
  (q, 011, XZ0)
  (q, 11, XXZ0)
  (p, 011, Z0)
  (q, 1, XXZ0)
  (p, 11, XZ0)
  (q, ε, XXZ0)
  (p, 1, XZ0)
  (p, 11, Z0)
  (p, 1, XXZ0)
  (p, ε, XZ0)
  (p, 1, Z0)
  (p, ε, XXZ0)
  (p, 1, ε)
  (p, ε, XXXZ0)
  (p, ε, Z0)
  (p, ε, ε)
```

```
w = 0101: 14 reachable IDs, accepted: yes
  (q, 0101, Z0)
  (q, 101, XZ0)
  (q, 01, XZ0)
  (p, 101, Z0)
  (q, 1, XXZ0)
  (p, 01, Z0)
  (p, 01, ε)
  (q, ε, XXZ0)
  (p, 1, XZ0)
  (p, ε, XZ0)
  (p, 1, Z0)
  (p, ε, XXZ0)
  (p, ε, Z0)
  (p, ε, ε)
```

```
w = 1010: 1 reachable IDs, accepted: no
  (q, 1010, Z0)
```

The language is L(P) = 0(0 + 1)∗: it must start with 0 (only q on Z0 reads 0 with Z0 on top) and then anything can follow — from p, 1s are pushed or the single Z0 absorbed (checked on all strings up to length 7). 1010 starts with 1: q has no move on 1 with Z0 on top, so only the initial ID exists.

(b) This is the ww^R PDA with a $ marker: push the first half, guess the middle (ε-move), pop-and-match, remove $ and accept. 1001: (q1,1001,ε) ⊢ (q2,1001,$) ⊢ (q2,001,1$) ⊢ (q2,01,01$) ⊢ (q3,01,01$) ⊢ (q3,1,1$) ⊢ (q3,ε,$) ⊢ (q4,ε,ε) accepted. 0101: after pushing 01 the match needs 0 against top 1 → no move; every other guess also dies — rejected. L = {ww^R | w ∈ {0,1}∗} (even palindromes; ε accepted since q1 is accepting).

```
Accepting computation on bbcbb:
(s, bbcbb, ε) ⊢ (p, bbcbb, Z) ⊢ (p, bcbb, bZ) ⊢ (p, cbb, bbZ) ⊢ (m, bb, bbZ) ⊢ (m, b, bZ) ⊢ (m, ε,
Z) ⊢ (
acc, ε, ε)
```

In Atif's notation: (s, bbcbb, Z0) ⊢ (p, bbcbb, Z0) ⊢ … the same sequence with stack written top-left.

Lesson 7 — Designing PDAs

Slides 20–55, 87–101 + TF design problems (see also Hakim Sir's PDA note)

1. Slide Examples: Balanced Parentheses and Equal a's/b's (sl. 87–101)

```
Balanced, starting with ( : P = ({q0,q1}, {(,)}, {(,),Z0}, δ, q0, Z0, {q1})
  δ(q0, (, Z0) = {(q0, (Z0)}    δ(q0, (, () = {(q0, (()}    δ(q0, ), () = {(q0, ε)}    δ(q0, ε, Z0) =
{(q1, Z0)}
  trace of ()(): (q0,()(),Z0) ⊢ (q0,)(),(Z0) ⊢ (q0,(),Z0) ⊢ (q0,),(Z0) ⊢ (q0,ε,Z0) ⊢ (q1,ε,Z0)
())→stuck, )()(→stuck

Equal a's and b's: Q = {q, f}, Γ = {a, b, Z0}
  (1) δ(q,a,Z0)={(q,aZ0)}  (2) δ(q,a,a)={(q,aa)}  (3) δ(q,a,b)={(q,ε)}
  (4) δ(q,b,Z0)={(q,bZ0)}  (5) δ(q,b,b)={(q,bb)}  (6) δ(q,b,a)={(q,ε)}  (7) δ(q,ε,Z0)={(f,Z0)}
  the stack holds the EXCESS: a's if more a's were read, b's if more b's; Z0 marks the bottom
```

2. Design Catalogue (all machines simulator-checked)

Notation below: 'a, X → Y' (Sipser). Atif's δ form: δ(q, a, X) ∋ (r, Y X′) where the pushed string replaces X; 'pop' = γ ε; 'push' = γ = YX.

```
Equal numbers of a's and b's (2015-16 Q6(a); 2009-10 Q10(a)):
     s ──ε, ε → Z──▶ q
     q ──a, Z → AZ──▶ q
     q ──a, A → AA──▶ q
     q ──a, B → ε──▶ q
     q ──b, Z → BZ──▶ q
     q ──b, B → BB──▶ q
     q ──b, A → ε──▶ q
     q ──ε, Z → ε──▶ acc
(label: input, pop → push; ε = nothing; Z = bottom marker; accept by final state with input consumed)
```

```
Palindromes over {0,1} (2006-07 Q9, 2010-11 Q13(a), 2009-10 Q10(a)):
     s ──ε, ε → Z──▶ p
     p ──0, ε → 0──▶ p
     p ──1, ε → 1──▶ p
     p ──ε, ε → ε──▶ m
     p ──0, ε → ε──▶ m
     p ──1, ε → ε──▶ m
     m ──0, 0 → ε──▶ m
     m ──1, 1 → ε──▶ m
     m ──ε, Z → ε──▶ acc
(label: input, pop → push; ε = nothing; Z = bottom marker; accept by final state with input consumed)
```

```
{ww^R | w ∈ {0,1}*} (2016-17 Q7(b)):
     s ──ε, ε → Z──▶ p
     p ──0, ε → 0──▶ p
     p ──1, ε → 1──▶ p
     p ──ε, ε → ε──▶ m
     m ──0, 0 → ε──▶ m
     m ──1, 1 → ε──▶ m
     m ──ε, Z → ε──▶ acc
(label: input, pop → push; ε = nothing; Z = bottom marker; accept by final state with input consumed)
```

```
{wcw^R | w ∈ {a,b}*} (2014-15 Q6(c)):
     s ──ε, ε → Z──▶ p
     p ──a, ε → a──▶ p
     p ──b, ε → b──▶ p
     p ──c, ε → ε──▶ m
     m ──a, a → ε──▶ m
     m ──b, b → ε──▶ m
     m ──ε, Z → ε──▶ acc
(label: input, pop → push; ε = nothing; Z = bottom marker; accept by final state with input consumed)
```

```
Odd length, middle symbol a (2011-12 Q3(c), 2012-13 Q4(c)):
     s ──ε, ε → Z──▶ p
     p ──a, ε → X──▶ p
     p ──b, ε → X──▶ p
     p ──a, ε → ε──▶ m
     m ──a, X → ε──▶ m
     m ──b, X → ε──▶ m
     m ──ε, Z → ε──▶ acc
(label: input, pop → push; ε = nothing; Z = bottom marker; accept by final state with input consumed)
```

```
{a^ib^jc^k | i + j = k} (2011-12 Q4(a), 2012-13 Q4(a)):
     s ──ε, ε → Z──▶ qa
    qa ──a, ε → X──▶ qa
    qa ──ε, ε → ε──▶ qb
    qb ──b, ε → X──▶ qb
    qb ──ε, ε → ε──▶ qc
    qc ──c, X → ε──▶ qc
    qc ──ε, Z → ε──▶ acc
(label: input, pop → push; ε = nothing; Z = bottom marker; accept by final state with input consumed)
```

```
{0^n1^m | n ≤ m ≤ 2n} (2016-17 Q6(a)): each 0 pushes one OR two X's (guess):
     s ──ε, ε → Z──▶ p
     p ──0, ε → X──▶ p
     p ──0, ε → XX──▶ p
     p ──ε, ε → ε──▶ q
     q ──1, X → ε──▶ q
     q ──ε, Z → ε──▶ acc
(label: input, pop → push; ε = nothing; Z = bottom marker; accept by final state with input consumed)
```

```
{a^ic^k(bc)^i | i, k ≥ 0} (2007-08 Q5):
     s ──ε, ε → Z──▶ qa
    qa ──a, ε → X──▶ qa
    qa ──ε, ε → ε──▶ qc
    qc ──c, ε → ε──▶ qc
    qc ──ε, ε → ε──▶ qb
    qb ──b, X → ε──▶ qx
    qx ──c, ε → ε──▶ qb
    qb ──ε, Z → ε──▶ acc
(label: input, pop → push; ε = nothing; Z = bottom marker; accept by final state with input consumed)
```

```
{a^nb^(2n) | n > 0} (2008-09 Q1(i)); for n > 0 drop the direct qa → qacc move:
    qs ──ε, ε → Z──▶ qa
    qa ──a, ε → X──▶ qa
    qa ──b, ε → ε──▶ qb1
   qb1 ──b, X → ε──▶ qb0
   qb0 ──b, ε → ε──▶ qb1
    qa ──ε, Z → ε──▶ qacc
   qb0 ──ε, Z → ε──▶ qacc
(label: input, pop → push; ε = nothing; Z = bottom marker; accept by final state with input consumed)
```

```
{a^ib^jc^k | i = j or i = k} (★★★ 2022-23 Q8, Jan-2020 Q8):
     s ──ε, ε → Z──▶ q
     q ──ε, ε → ε──▶ p1
     q ──ε, ε → ε──▶ r1
    p1 ──a, ε → X──▶ p1
    p1 ──ε, ε → ε──▶ p2
    p2 ──b, X → ε──▶ p2
    p2 ──ε, Z → Z──▶ p3
    p3 ──c, ε → ε──▶ p3
    p3 ──ε, ε → ε──▶ f1
    r1 ──a, ε → X──▶ r1
    r1 ──ε, ε → ε──▶ r2
    r2 ──b, ε → ε──▶ r2
    r2 ──ε, ε → ε──▶ r3
    r3 ──c, X → ε──▶ r3
    r3 ──ε, Z → ε──▶ f2
(label: input, pop → push; ε = nothing; Z = bottom marker; accept by final state with input consumed)
```

```
{a^ib^jc^k | i = j or j = k} (2013-14 Q1(b)):
     s ──ε, ε → Z──▶ q
     q ──ε, ε → ε──▶ p1
     q ──ε, ε → ε──▶ r1
    p1 ──a, ε → X──▶ p1
    p1 ──ε, ε → ε──▶ p2
    p2 ──b, X → ε──▶ p2
    p2 ──ε, Z → Z──▶ p3
    p3 ──c, ε → ε──▶ p3
    p3 ──ε, ε → ε──▶ f1
    r1 ──a, ε → ε──▶ r1
    r1 ──ε, ε → ε──▶ r2
    r2 ──b, ε → Y──▶ r2
    r2 ──ε, ε → ε──▶ r3
    r3 ──c, Y → ε──▶ r3
    r3 ──ε, Z → ε──▶ f2
(label: input, pop → push; ε = nothing; Z = bottom marker; accept by final state with input consumed)
```

```
{a^nb^nc^md^m | n, m ≥ 1} (★★★ 2017-18 Q8):
     s ──ε, ε → Z──▶ qa
    qa ──a, ε → X──▶ qa
    qa ──b, X → ε──▶ qb
    qb ──b, X → ε──▶ qb
    qb ──c, Z → YZ──▶ qc
    qc ──c, ε → Y──▶ qc
    qc ──d, Y → ε──▶ qd
    qd ──d, Y → ε──▶ qd
    qd ──ε, Z → ε──▶ acc
(label: input, pop → push; ε = nothing; Z = bottom marker; accept by final state with input consumed)
```

```
{a^nb^mc^md^n | n, m ≥ 1} (★★★ 2023-24 Q8):
     s ──ε, ε → Z──▶ qa
    qa ──a, ε → A──▶ qa
    qa ──b, A → BA──▶ qb
    qb ──b, ε → B──▶ qb
    qb ──c, B → ε──▶ qc
    qc ──c, B → ε──▶ qc
    qc ──d, A → ε──▶ qd
    qd ──d, A → ε──▶ qd
    qd ──ε, Z → ε──▶ acc
(label: input, pop → push; ε = nothing; Z = bottom marker; accept by final state with input consumed)
```

3. ARITH: Legal Arithmetic Expressions (★★★ 2021-22 Q7, 2019-20 Q8)

ARITH = strings over {a, +, ∗, (,)} generated by E → E + E | E ∗ E | (E) | a. Stack idea: the stack records the number of open parentheses. States: E = 'an operand is expected' (start), O = 'an operator or a closing parenthesis may follow'. In E: read a → O; read (→ push X, stay in E. In O: read + or ∗ → E; read) with X on top → pop, stay in O. Accept in O when only Z is left. (Checked against the grammar by tree counting on all strings up to length 5.)

```
PDA for ARITH:
     s ──ε, ε → Z──▶ E
     E ──a, ε → ε──▶ O
     E ──(, ε → X──▶ E
     O ──+, ε → ε──▶ E
     O ──*, ε → ε──▶ E
     O ──), X → ε──▶ O
     O ──ε, Z → ε──▶ acc
(label: input, pop → push; ε = nothing; Z = bottom marker; accept by final state with input consumed)
```

```
Accepting computation on (a+a)*a:
(s, (a+a)*a, ε) ⊢ (E, (a+a)*a, Z) ⊢ (E, a+a)*a, XZ) ⊢ (O, +a)*a, XZ) ⊢ (E, a)*a, XZ) ⊢ (O, )*a, XZ)
⊢ (O,
 *a, Z) ⊢ (E, a, Z) ⊢ (O, ε, Z) ⊢ (acc, ε, ε)
```

Exam Approach — PDA Design

1. State the stack strategy first; choose the stack symbols and what each means.
2. Push Z0/Z first (ε-move) so you can detect 'stack empty'; accept by moving into the final state on Z.
3. Unions (or) → nondeterministic branch at the start; unmarked boundaries → ε-guess between phases.
4. Show acceptance of the string the question names (and rejection of the other) by configurations.

TRAP Remember n ≥ 1 vs n ≥ 0 (the direct 'empty' move), pop symbols must be checked (b, X pops only X), and a branch for 'i = j' must let the c's pass without touching the stack.

Practice — Lesson 7

(a) Actual — 2016-17 Q6(a). (b) Actual — 2022-23 Q8. (c) Actual — 2023-24 Q8. (d) Actual — 2017-18 Q8. (e) Actual — 2021-22 Q7. (f) Actual — 2012-13 Q4(a),(c). (g) Actual — 2007-08 Q5.

(a) Design a PDA for L = {0^n1^m | n ≤ m ≤ 2n}. Provide the transition diagram (no need for the function). Show that it accepts 00111 and rejects 0011111. (10 + 5)

(b) PDA for {a^ib^jc^k | i, j, k ≥ 0 and (i = j or i = k)}; transition diagram. (15)

(c) PDA for {a^nb^mc^md^n | n ≥ 1, m ≥ 1}; diagram. (15)

(d) PDA for {a^nb^nc^md^m | n ≥ 1, m ≥ 1}; diagram. (15)

(e) Design a PDA for ARITH. (15)

(f) PDAs for {a^ib^jc^k : i + j = k} and for odd-length strings over {a,b} with middle symbol a. (8 + 8⅓)

(g) PDA for {a^ic^k(bc)^i | i, k ≥ 0}. (10)

✓ Solution

(a) Strategy: nondeterministically push ONE or TWO X per 0; pop one X per 1; accept when the stack is back to Z at the end. Then m = (number of X) is between n and 2n.

```
Accepting computation on 00111:
(s, 00111, ε) ⊢ (p, 00111, Z) ⊢ (p, 0111, XZ) ⊢ (p, 111, XXXZ) ⊢ (q, 111, XXXZ) ⊢ (q, 11, XXZ) ⊢
(q, 1, X
Z) ⊢ (q, ε, Z) ⊢ (acc, ε, ε)
```

00111: guess pushes 1, 2 (three X's), then three 1s pop them ✓. 0011111: five 1s, but at most 4 X's can be pushed by two 0s — after four pops the fifth 1 has no X on top: rejected ✓.

(b)–(g) Machines and traces are listed in the catalogue above (each verified). For the diagrams label arcs 'input, pop/push' exactly as in the listings; start with ε, ε/Z.

Lesson 8 — Final State vs Empty Stack

Slides 102–109

$$L(P) = \{ w \mid (q0, w, Z0) \vdash^* (q, \varepsilon, \alpha), q \in F \}$$
$$N(P) = \{ w \mid (q0, w, Z0) \vdash^* (q, \varepsilon, \varepsilon) \} \quad \text{(F irrelevant)}$$

MEMORIZE A language has a PDA accepting by final state iff it has one accepting by empty stack iff it is context-free (sl. 102–104). The two conversions both use a new bottom marker X0 and a new start state p0.

1. Empty Stack → Final State (sl. 106–107)

Given PN = (Q, Σ, Γ, δ, q0, Z0, F): PF = (Q ∪ {p0, pf}, Σ, Γ ∪ {X0}, δF, p0, X0, {pf}) with δF(p0, ε, X0) = {(q0, Z0X0)} (start PN on top of the new marker), all moves of δ kept, and δF(q, ε, X0) ∋ (pf, ε) for every q ∈ Q. PN empties its stack on w ⇔ X0 is exposed ⇔ PF moves to pf.

2. Final State → Empty Stack (sl. 108–109)

Given PF = (Q, Σ, Γ, δ, q0, Z0, F): PN = (Q ∪ {p0, p}, Σ, Γ ∪ {X0}, δN, p0, X0) with δN(p0, ε, X0) = {(q0, Z0X0)}; keep δ; for every f ∈ F and every stack symbol Y (including X0): δN(f, ε, Y) ∋ (p, ε); and δN(p, ε, Y) = {(p, ε)} for all Y (drain the stack). The marker X0 prevents PF from emptying its stack accidentally before acceptance.

Checked on a concrete machine: PF for 0^n1^n (n ≥ 1, final state q2) and its PN built as above accept exactly the same strings (all strings up to length 8), and an empty-stack PDA for balanced parentheses converted to final state accepts the same strings.

Practice — Lesson 8

(a) Actual — 2007-08 Q6, 2019-20 Q10(a). (b) Actual — 2015-16 Q6(b). (c) Actual — 2009-10 Q10(b)(ii).

(a) Justify: a language L has a PDA accepting it by final state iff it has a PDA accepting it by empty stack. (8–10)

(b) A PDA PF accepts L by final state. Is it always possible to construct PN accepting L by empty stack? If so show the construction with definitions of PF and PN. (10⅓)

(c) Convert the PDA of 2009-10 Q10(b) to one accepting the same language by empty stack.

✓ Solution

(a)–(b) Both directions are the constructions of §1–§2; correctness: in PN built from PF, a computation reaches p only from an accepting state of PF (so w ∈ L(PF)), then drains the stack — hence w ∈ N(PN); conversely if w ∈ N(PN) the only way X0 can be popped is through p, so PF accepted. For §1 the argument is symmetric.

(c) With the PDA of 2009-10 Q10(b) (accepting state p, start q, Z the start symbol): new start p0, new bottom X0; δ′(p0, ε, X0) = {(q, ZX0)}; keep (i)–(vii); add δ′(p, ε, Y) = {(r, ε)} for Y ∈ {Z, X, X0} and δ′(r, ε, Y) = {(r, ε)}, with r a new draining state.

Lesson 9 — CFG ↔ PDA

Slides 104, 110–122

MEMORIZE CFLs = languages of CFGs = languages accepted by PDAs by final state = by empty stack (sl. 104).

1. Grammar → PDA (sl. 110–115)

A PDA simulating leftmost derivations: for S ⇒∗ xAα ⇒∗ w it has matched x with input and keeps Aα on the stack. Given G = (V, T, P, S): P = ({q}, T, V ∪ T, δ, q, S) accepting by empty stack with (1) δ(q, ε, A) = {(q, β) | A → β ∈ P} for each variable A; (2) δ(q, a, a) = {(q, ε)} for each terminal a. Example a + a ∗ b: E ⇒ E + E ⇒ I + E ⇒ a + E ⇒ … ⇒ a + a ∗ b is mirrored by (q, a+a∗b, E) ⊢ (q, a+a∗b, E+E) ⊢ (q, a+a∗b, I+E) ⊢ (q, a+a∗b, a+E) ⊢ (q, +a∗b, +E) ⊢ … ⊢ (q, ε, ε) (sl. 112–113). Theorem N(P) = L(G) by induction on derivation length (sl. 114–115, proof skipped).

2. PDA → Grammar (sl. 116–122)

Variables [pXq] mean: 'starting in state p with X on top, the PDA pops X (net) after reading some input, ending in q'. G = (V, Σ, R, S): S → [q0Z0p] for every state p; if δ(q, a, X) ∋ (r, Y1Y2…Yk) then for EVERY list of states r1, …, rk: [q X rk] → a [r Y1 r1][r1 Y2 r2] … [r(k−1) Yk rk]; for k = 0 the rule is [qXr] → a. Many variables are useless — delete non-generating ones, then unreachable ones.

```
Parentheses (δ(q0,(,Z0)={(q0,(Z0)}, δ(q0,(,()={(q0,(()}, δ(q0,),()={(q0,ε)}, δ(q0,ε,Z0)={(q1,ε)}) —
pruned grammar:
S → [q0Z0q1]
[q0Z0q1] → ( [q0(q0] [q0Z0q1]  |  ε
[q0(q0]  → ( [q0(q0] [q0(q0]  |  )
```

Renaming A = [q0Z0q1], B = [q0(q0]: S → A, A → (BA | ε, B → (BB |) — the balanced-parentheses grammar (checked on all strings up to length 8).

Exam Approach

1. Grammar → PDA: one state, a transition per production (pop A, push the body), one per terminal (match and pop). Always accept by empty stack; mention how to convert to final state if asked.
2. PDA → grammar: write all δ entries, generate the triple rules mechanically, then prune: a variable [pXq] is generating only if some rule for it has all-generating variables; keep only variables reachable from S.
3. Say what [pXq] means in your answer.

Practice — Lesson 9

(a) Actual — 2009-10 Q11(a). (b) Actual — 2012-13 Q1(a). (c) Actual — 2011-12 Q4(b). (d) Actual — 2013-14 Q1(a). (e) Actual — 2010-11 Q12(d). (f) Actual — 2009-10 Q11(b). (g) Actual — 2010-11 Q13(b). (h) Actual — 2013-14 Q1(c), 2014-15 Q7(a), 2012-13 Q1(b), 2010-11 Q12(c).

(a) Convert S → AS | A, A → 0A | 1B | 1, B → 0B | 0 to a PDA accepting by empty stack. (5)

(b) PDA for S → BS | A, A → 0A | ε, B → BB1 | 2. (10)

(c) Convert S → aTXb, T → XTS | ε, X → a | b to a PDA. (8)

(d) Convert E → E + T | T, T → T × F | T/F | F, F → (E) | a to a PDA. (7)

(e) Informally describe and draw a PDA for S → TX, T → 0T0 | 1T1 | #X, X → 0X | 1X | ε. (5⅓)

(f) Convert the PDA P = ({p, q}, {0,1}, {Z, X}, δ, q, Z) with δ(q,1,Z) = {(q,XZ)}, δ(q,0,X) = {(p,X)}, δ(p,1,X) = {(p,ε)} to a context-free grammar. (13)

(g) Convert the palindrome PDA of 2010-11 Q13(a) to an equivalent CFG. (7⅓)

(h) Prove that for any PDA P there is a CFG G with L(P) = L(G); and that a language is CF iff some PDA accepts it. (7–13)

✓ Solution

(a) P = ({q}, {0,1}, {S, A, B, 0, 1}, δ, q, S), accepting by empty stack:

```
δ(q,ε,S) = {(q,AS), (q,A)}
δ(q,ε,A) = {(q,0A), (q,1B), (q,1)}
δ(q,ε,B) = {(q,0B), (q,0)}
δ(q,0,0) = {(q,ε)}    δ(q,1,1) = {(q,ε)}
```

(Verified: this PDA accepts exactly L(G) on all strings up to length 6.)

(b) Same recipe: δ(q,ε,S) = {(q,BS),(q,A)}; δ(q,ε,A) = {(q,0A),(q,ε)}; δ(q,ε,B) = {(q,BB1),(q,2)}; δ(q,0,0), δ(q,1,1), δ(q,2,2) = {(q,ε)}. Note the left-recursive rule B → BB1 is fine for a PDA (it only pushes). Start symbol S; empty-stack acceptance.

(c) δ(q,ε,S) = {(q,aTXb)}; δ(q,ε,T) = {(q,XTS),(q,ε)}; δ(q,ε,X) = {(q,a),(q,b)}; δ(q,a,a) = δ(q,b,b) = {(q,ε)}.

(d) δ(q,ε,E) = {(q,E+T),(q,T)}; δ(q,ε,T) = {(q,T×F),(q,T/F),(q,F)}; δ(q,ε,F) = {(q,(E)),(q,a)}; and δ(q,t,t) = {(q,ε)} for t ∈ {a, +, ×, /, (,)}. (13 transitions, verified.)

(e) Informal: T pushes a copy of w and its mirror requirement as it goes (0T0 / 1T1 are expanded on the stack), after # the X part is free. Formally δ(q,ε,S) = {(q,TX)}; δ(q,ε,T) = {(q,0T0),(q,1T1),(q,#X)}; δ(q,ε,X) = {(q,0X),(q,1X),(q,ε)}; δ(q,0,0), δ(q,1,1), δ(q,#,#) = {(q,ε)}. The language is w # (stuff) with the mirror rules 0T0 | 1T1: L = {w # u w^R v | w, u, v ∈ {0,1}∗} (T generates w # u w^R, X appends v; verified against the grammar).

(f) L(P) = ∅: nothing ever pops Z, so the stack is never empty, N(P) = ∅. The construction confirms it: variables S → [qZq] | [qZp]; [qZr2] → 1[qXr1][r1Zr2] — but every [r1 Z r2] needs a move that pops Z and there is none, so no [·Z·] variable is generating; [qXp] → 0[pXp] → 01 is generating but useless. After pruning, the grammar has no rule for S: L(G) = ∅ (checked by program). Hence the answer 'S has no productions' (or equivalently the empty grammar) is correct.

(g) Empty-stack version of the palindrome PDA (states p = pushing, m = matching):

```
δ(p,a,Z) = {(p,aZ)}, δ(p,a,X) = {(p,aX)} for a ∈ {0,1}, X ∈ {0,1}; δ(p,ε,Z) = {(m,ε)}; δ(p,ε,X) =
{(m,X)}
δ(m,0,0) = δ(m,1,1) = {(m,ε)}; δ(m,ε,Z) = {(m,ε)}
CFG (after pruning; [pZm] = S'):  S → [pZm]
  [pZm] → 0[p0m][mZm] | 1[p1m][mZm] | ε
  [p0m] → 0[p0m][m0m] | 1[p1m][m0m] | [m0m]      [p1m] → 0[p0m][m1m] | 1[p1m][m1m] | [m1m]
  [m0m] → 0      [m1m] → 1      [mZm] → ε
```

It generates exactly the even-length palindromes (checked up to length 8).

(h) (⇒ CF) Grammar → PDA: §1; (⇐) PDA → grammar: §2. Proof sketch of §2 — claim [qXp] ⇒∗ w iff (q, w, X) ⊢∗ (p, ε, ε). (⇐) induction on the number of steps: the first move reads a (or ε), replaces X by Y1…Yk, and the remaining moves split w = w1…wk where each wi pops Yi, moving r(i−1) → ri; by induction [r(i−1) Yi ri] ⇒∗ wi, so the rule gives [qXp] ⇒∗ a w1…wk. (⇒) induction on the derivation length, reading the first rule backwards. With the start rules S → [q0Z0p]: S ⇒∗ w iff (q0, w, Z0) ⊢∗ (p, ε, ε) iff w ∈ N(P). Since final-state and empty-stack PDAs accept the same languages (Lesson 8), L(P) is context-free.

Lesson 10 — CFL Pumping Lemma, DPDA and the Hierarchy

Slides 123–136

1. The Pumping Lemma for CFLs (sl. 123–129)

MEMORIZE L CFL ⇒ ∃ n such that every z ∈ L with |z| ≥ n can be written z = uvwxy with (1) |vwx| ≤ n, (2) vx ≠ ε, (3) uv^iwx^iy ∈ L for all i ≥ 0.

Non-CF examples: {0^n1^n2^n} (cannot match three groups), {0^i1^j2^i3^j} (two interleaved pairs), {ww} (two arbitrary strings over an alphabet of size > 1) (sl. 123).

EXAM-READY ANSWER Proof (2022-23 Q9; sl. 126–129). Lemma (parse-tree size, CNF): if the longest path of a CNF parse tree has n edges then |yield| ≤ 2^(n−1) (induction: n = 1 gives one terminal; a root with two subtrees of height ≤ n − 1 gives ≤ 2·2^(n−2) = 2^(n−1)). Let G be a CNF grammar for L (minus ε) with m variables and take the pumping length 2^m. Let |z| ≥ 2^m. A tree whose longest path has ≤ m edges has yield ≤ 2^(m−1) = n/2 < |z|, so the tree for z has a path of length ≥ m + 1 edges, i.e. ≥ m + 2 nodes of which the last m + 1 are variables A0, A1, …, Ak (k ≥ m). Only m different variables exist, so Ai = Aj = A for some i < j among the last m + 1. Take the lowest such pair: the subtree at Ai has height ≤ m + 1 hence yield |vwx| ≤ 2^m = n. Write the yield of Ai as vwx and of Aj as w; z = uvwxy. Since Ai ≠ Aj as nodes and the grammar is CNF, the first step from Ai goes to two children, one of which leads to Aj and the other contributes a non-empty string, so vx ≠ ε. Replacing the subtree at Ai by the one at Aj gives uwy (i = 0); replacing the subtree at Aj by the one at Ai gives uv²wx²y; repeating gives all uv^iwx^iy ∈ L.

Standard application (sl. 125): {0^n1^n2^n}: take z = 0^n1^n2^n; |vwx| ≤ n so vwx cannot contain both 0s and 2s. If it has no 2s, uwy has fewer 0s or 1s than 2s; if it has no 0s, uwy has n 0s but fewer 1s or 2s — contradiction.

2. Deterministic PDAs (sl. 130–133)

A PDA is deterministic iff δ(q, a, X) has at most one member for every q, a ∈ Σ ∪ {ε}, X, and if δ(q, a, X) is non-empty for some a ∈ Σ then δ(q, ε, X) is empty. DPDAs accepting by empty stack have the prefix property (no member is a proper prefix of another) so they are weak; DPDAs accepting by final state recognise the **DCFLs**, which properly include the regular languages and are properly included in the CFLs; their languages have unambiguous grammars (but not every unambiguous CFL is a DCFL). ww^R has no DPDA; wcw^R (centre marker c) does (sl. 132). LR(k) grammars — used in compilers — generate exactly the DPDA languages (sl. 134).

3. Chomsky Hierarchy (sl. 135–136)

Class	Grammar	Machine
Regular	right-linear A → aB | a	DFA / NFA
Context-free	A → γ	PDA
Context-sensitive	αAβ → αγβ	linear bounded automaton
Recursively enumerable	unrestricted	Turing machine

Exam Approach — 'Show L Is Not Context-Free'

1. Choose z with every block of length n (or a multiple).

2. |vwx| ≤ n ⇒ vwx touches at most two adjacent blocks.
3. Case 1: v or x contains two symbol types → order destroyed when pumped. Case 2: v, x each inside one block → pumping changes at most two blocks, the third is fixed.
4. Pick i (usually 0 or 2), conclude.

Practice — Lesson 10

(a) Actual — 2012-13 Q2(b), 2016-17 Q7(c). (b) Actual — 2013-14 Q2(c). (c) Actual — 2014-15 Q6(b). (d) Actual — 2008-09 Q1(ii). (e) Actual — 2023-24 Q7, 2021-22 Q8. (f) Actual — 2022-23 Q9. (g) Actual — 2009-10 Q11(c).

(a) State the CFL pumping lemma and show {a^nb^nc^n | n ≥ 0} is not context-free. (13⅓ / 10)

(b) Use the pumping lemma to show {0^n1^n0^n1^n | n ≥ 0} and {0^n#0^(2n)#0^(3n) | n ≥ 0} are not context-free. (9⅓)

(c) Show {a^nb^jc^k : k = jn} is not context-free. (7)

(d) Is there a PDA for {a^nb^(2n)c^n | n ≥ 0}? Design one or prove none exists. (13⅓)

(e) Give an example of a non-context-free language and prove it with the pumping lemma. (15)

(f) State and prove the pumping lemma for CFLs, given the CNF parse-tree height theorem. (15)

(g) Briefly describe deterministic PDAs with an appropriate example. (5⅓)

✓ Solution

(a) Statement: MEMORIZE box. z = a^nb^nc^n; case analysis as in the Exam Approach (details: Hakim Lesson 21 and sl. 125). Not CF.

(b) (i) z = 0^n1^n0^n1^n: |vwx| ≤ n lies within two adjacent blocks. Pumping i = 2 changes the lengths of at most two adjacent blocks, so the counts of blocks 1 and 3 (or 2 and 4) become unequal, or the 0/1 order breaks. (ii) z = 0^n#0^(2n)#0^(3n): vwx contains at most one # (it has length ≤ n < 2n) so pumping either changes the number of #s (v or x contains #, i = 0 or 2 changes it) or changes at most two of the three zero-blocks, leaving the relation (n, 2n, 3n) broken.

(c) Assume CF with pumping length p; z = a^pb^pc^(p²) ∈ L (n = p, j = p, k = p·p). |vwx| ≤ p. Case 1: vx contains no c. Then #c = p² stays; if only a's (or only b's) are pumped with i = 2 the product (p+α)·p (or p·(p+β)) ≠ p²; if both, (p+α)(p+β) > p². Case 2: vwx contains c's. It cannot reach the a's (the p b's lie between), so #a = p. If vx has no b: pump i = 0, #c = p² − γ with 1 ≤ γ ≤ p, but p·#b = p² — mismatch. If vx has b's (β ≥ 1 of them, γ c's, β + γ ≤ p): with i = 0, p(p − β) = p² − γ would need γ = pβ ≥ p, impossible since γ ≤ p − β < p. In every case uv^iwx^iy ∉ L. Not CF.

(d) No PDA exists: {a^nb^(2n)c^n} is not CF — z = a^pb^(2p)c^p; vwx touches ≤ 2 adjacent blocks; i = 2 breaks either the order or the counts (#a = #c or #b = 2#a). (The PDA for a^nb^(2n) alone exists, Lesson 7.)

(e) {a^nb^nc^n}, proof as (a). Another: {ww | w ∈ {0,1}∗}.

(f) EXAM-READY ANSWER in §1.

(g) §2 with the example: wcw^R is a DCFL (push until c, then pop-and-match — no guess needed); ww^R is CF but not deterministic.

Quick Notes — PDA

Revision points for Lessons 5–10.

L5–L6 · Basics

- PDA = ε-NFA + stack; P = (Q, Σ, Γ, δ, q0, Z0, F); δ(q, a, X) ∋ (p, γ).
- ID (q, w, γ), top left; (q, aw, Xβ) ⊢ (p, w, αβ); no mixing and matching.
- Given PDA 2015-16: language 0(0+1)∗; list reachable IDs breadth-first.

L7 · Designs

- push Z first; accept by ε-move on Z; unions = branches; boundary unknown = ε-guess.
- i=j or i=k: two branches; a^nb^mc^md^n: A's then B's; a^nb^nc^md^m: two counters in sequence.
- n ≤ m ≤ 2n: push one or two X per 0. ARITH: states E (operand) / O (operator), stack = open '('.

L8 · FS ↔ ES

- New start p0 and marker X0 in both directions; FS→ES: accepting states drain the stack; ES→FS: ε,X0 → pf.

L9 · CFG ↔ PDA

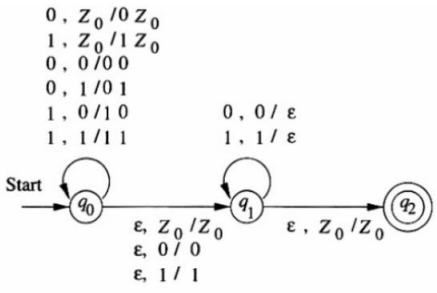
- CFG→PDA: one state; (ε, A) → bodies; (a, a) → ε; empty stack.
- PDA→CFG: [qXrk] → a[rY1r1]…[r(k−1)Ykrk] for all state lists; S → [q0Z0p]; prune.

L10 · Pumping, DPDA

- uvwxy, |vwx| ≤ n, vx ≠ ε, pump both; proof via CNF height (yield ≤ 2^(h−1)), n = 2^m.
- DPDA: ≤ 1 move, no ε/input conflict; DCFL ⊂ CFL; ww^R has no DPDA, wcw^R does.

Key Diagrams to Reproduce

The PDA of previous example is represented by the diagram shown.



Representing a PDA as a generalized transition diagram

ww^R PDA diagram (sl. 49)